

Лабораторная работа 1.

Решение задач первичного разведочного анализа данных и статистической проверки гипотез

1. Цель лабораторной работы

- получить навыки применения методов первичного разведочного анализа данных и статистической проверки гипотез для решения задач анализа данных (АД);
- получить базовые навыки работы в среде R;
- изучить средства R для проведения первичного разведочного анализа данных (методы визуализации, описательной статистики, статистической проверки гипотез) на примере решения конкретной задачи АД.
- изучить средства R для исследования взаимосвязей признаков (методы корреляционного, рангового корреляционного, дисперсионного анализа, анализа таблиц сопряженности на примере решения конкретной задачи АД).

2. Задание к лабораторной работе

Прочитайте содержательную постановку задачи для вашего варианта (в конце файла). Выберите файл с данными (в формате .csv), соответствующий вашему варианту.

1. Загрузите файл с данными, соответствующий варианту.
2. Просмотрите загруженную таблицу с данными. Попробуйте использовать команды R для доступа к определенному столбцу/строке данных, редактирования данных, получения подвыборок из данных в соответствии с заданными условиями.
3. Выполните переименование признаков, в случае необходимости (рекомендуется не давать длинные названия признакам для удобства работы и представления результатов анализа).
4. Посмотрите структуру данных. Определите тип каждого признака в данных (количественный, порядковый (ординальный, ранговый), классификационный (номинальный)). Для порядковых и классификационных признаков задайте тип factor.
5. Рассчитайте основные статистические характеристики по количественным признакам (минимальное, максимальное, среднее значение, стандартное отклонение, дисперсия, первый и третий квартили, медиана, мода, асимметрия, эксцесс) отдельно для первой и второй групп и для всей выборки. Сделайте содержательные выводы.
6. Проведите графический анализ данных, постройте:
 - диаграмму рассеяния по двум любым количественным признакам;
 - радиальную диаграмму по качественному признаку;

- категориальную радиальную диаграмму по одному из качественных признаков в зависимости от пола и группы;
- категориальную столбиковую диаграмму по одному из количественных признаков в зависимости от пола и группы;
- диаграмму размаха для одного из количественных признаков в зависимости от значений пола и/или группы;
- гистограммы для всех количественных признаков в одном графическом окне, для каждого признака отдельный график;
- матричный график по всем количественным признакам.

На основе проведенного анализа сделайте выводы о структуре данных, о характере распределения данных в терминах решаемой задачи.

7. Проведите корреляционный анализ данных для количественных признаков

7.1. Проверьте гипотезу о соответствии данных модели нормального закона распределения на основе критериев согласия χ^2 (хи-квадрат), Шапиро – Уилка, Крамера – Мизеса. Проверку выполните для одного из количественных признаков. Приведите графическую иллюстрацию (гистограмма распределения значений признака с наложением функции НЗР). Сделайте выводы в терминах прикладной задачи о типе распределения данных.

7.2. Оцените степень взаимосвязи между всеми количественными признаками на основе расчета коэффициентов корреляции Пирсона, Спирмена, Кендалла для первой и второй группы.

7.3. Оцените статистическую значимость коэффициента корреляции Пирсона для одной пары признаков на основе критерия Стьюдента. Сделайте выводы в терминах прикладной задачи о статистической значимости связи между признаками.

7.4. Оцените степень взаимосвязи между количественными признаками на основе расчета частного коэффициента корреляции Пирсона для первой и второй группы. Сделайте выводы в терминах прикладной задачи о статистической значимости связи между признаками.

7.5. Графически представьте матрицу коэффициентов корреляции с наложением тепловой карты.

7.6. Сделайте общие выводы о силе и направлении связи между количественными признаками в терминах решаемой задачи.

8. Оцените степень взаимосвязи между одним из качественных признаков и количественными признаками на основе использования однофакторного дисперсионного анализа (ANOVA) и критерия Краскела-Уоллиса. Сделайте выводы в терминах прикладной задачи о статистической значимости связи между признаками.

9. Выполните анализ таблиц сопряженности. Оцените степень взаимосвязи между качественными признаками на основе критериев χ^2 (Chi-квадрат) и Фишера. Сделайте выводы в терминах прикладной задачи о статистической значимости связи между признаками.

10. Постройте матричный график с помощью функций пакета `ggpairs`. При построении графика используйте качественные и количественные признаки. Опишите график, сделайте выводы.

11. Сделайте выводы по работе в произвольной форме.

3. Методические указания к лабораторной работе

R – это специализированный язык программирования и среда для решения задач статистической обработки данных. R работает во всех основных операционных системах и поддерживает тысячи специализированных модулей, библиотек и утилит. Все это делает R удобным средством для извлечения полезной информации из гор сырых данных.

R можно бесплатно скачать из сетевого архива R (Comprehensive R Archive Network, CRAN) по адресу <http://cran.r-project.org>. Предварительно скомпилированные загрузочные файлы доступны для Linux, macOS (ранее Mac OS X) и Windows.

Удобная интегрированная среда разработки для R – RStudio, которая наиболее широко используется в настоящее время. Чтобы установить RStudio, необходимо загрузить последнюю версию установщика для вашей платформы с сайта RStudio (<https://rstudio.com/products/rstudio/download/>). Версию RStudio Desktop с открытым исходным кодом можно скачать и использовать бесплатно. Новые версии R и RStudio выпускаются каждые несколько месяцев. Обратите внимание, что RStudio работает с любой версией R, которую вы установили, поэтому обновление до последней версии RStudio не обновляет вашу версию R. R необходимо обновлять отдельно.

R – это чувствительный к регистру клавиатуры интерпретируемый язык программирования. Вы можете либо вводить команды по одной в ответ на приглашение на ввод команды (`>`), либо запускать набор команд из исходного файла. Типы данных очень разнообразны: векторы, матрицы, таблицы данных и списки (совокупность нескольких объектов) (см. Кабаков, 2023, с. 58-72).

Основной функционал программы реализуется при помощи встроенных и создаваемых пользователем функций, и все объекты (наборы данных) хранятся в памяти программы до завершения интерактивной сессии. Основные функции доступны по умолчанию. Остальные функции содержатся в пакетах, которые могут активироваться в ходе работы с программой по мере необходимости.

Командные строки состоят из функций и присвоений. R использует символ `<-` для присвоений вместо обычного знака равенства. Например, командная строка `x <- rnorm(10)` создает объект типа вектор, который называется `x` и содержит десять случайных элементов нормального распределения.

Также в R можно использовать знак равенства для присвоения, но этот синтаксис поддерживается только некоторыми функциями и не является стандартным.

Комментарии в R начинаются с символа #.

R в базовой установке уже обладает обширными возможностями. Однако некоторые наиболее впечатляющие опции программы реализованы в дополнительных модулях, которые можно скачать и установить. По состоянию на сентябрь 2026 года существует более 24 800 созданных пользователями модулей, называемых *пакетами* (packages), которые вы можете скачать с <http://cran.r-project.org/web/packages> и пакеты добавляются ежедневно.

Пакеты – это собрания функций R, данных и скомпилированного программного кода в определенном формате. Директория, в которой пакеты хранятся на вашем компьютере, называется библиотекой. Функция `.libPath()` показывает, где расположена ваша библиотека, а функция `library()` выводит на экран названия всех имеющихся в библиотеке пакетов.

R поставляется уже со стандартным набором пакетов (включая `base`, `datasets`, `utils`, `grDevices`, `graphics` и `methods`). В них уже содержатся разнообразные функции и наборы данных, доступных по умолчанию. Другие пакеты нужно скачивать и устанавливать. После установки они загружаются во время сессии по мере необходимости. Команда `search()` выводит на экран названия загруженных и готовых к использованию пакетов.

В R существует множество функций, предназначенных для управления пакетами. Для установки пакета используйте команду

```
>install.packages()
```

Эта команда, введенная без аргументов, вызовет список зеркал сайта CRAN. Выберите ближайший к вам сайт или самый первый, указанный в верхней части списка (0-Cloud), который, как правило, хорошо работает для большинства местоположений. После выбора зеркала вы увидите список всех доступных пакетов. Выберите один из них, и он будет скачан и загружен.

Обратите внимание! Не все зеркала могут быть доступны и потребуются выбор другого зеркала.

Для выбора зеркала можно использовать команду:

```
>chooseCRANmirror()
```

Если вы знаете название пакета, который хотите установить, то сообщите функции это название в виде аргумента. Например, пакет `gclus` содержит функции для создания усовершенствованных диаграмм рассеяния. Этот пакет можно скачать и установить при помощи команды `install.packages("gclus")`. Пакет нужно установить только один раз. Однако, как и любые другие программы, пакеты часто обновляются их разработчиками. Используйте команду `update.package()` для обновления всех установленных пакетов. Для получения подробной информации об установленных пакетах можно использовать команду `installed.packages()`. Она выводит на экран список всех имеющихся пакетов с номерами их версий, названиями пакетов, от которых они зависят, и другой информацией.

Когда вы устанавливаете пакет, он скачивается с сайта CRAN и загружается в вашу библиотеку. Для использования этого пакета в текущей сессии

программы вам нужно загрузить его при помощи команды `library()`. Например, для того чтобы использовать пакет `gclus`, введите команду:

```
> library(gclus).
```

Задав значение переменной `.libPaths`, можно сформировать локальную библиотеку пакетов вне системы директорий R. Это может пригодиться для создания резервных копий при обновлении пакетов.

Пример:

```
>.libPaths("C:\\Users\\AppData\\Local\\Temp\\RtmpqiArKq\\downloaded_packages")
```

R обладает обширными справочными материалами. Встроенная система помощи содержит подробные разъяснения, ссылки на литературу и примеры для каждой функции из установленных пакетов. Справку можно вызвать при помощи функций, перечисленных в таблице 1.

Таблица 1 – Функции справки

Функция	Действие
<code>help.start()</code>	Общая справка
<code>help("нечто")</code> или <code>?нечто</code>	Справка по функции <i>нечто</i> (кавычки необязательны)
<code>help.search("нечто")</code> или <code>??нечто</code>	Поиск в справке записей, содержащих <i>нечто</i>
<code>example("нечто")</code>	Примеры использования функции <i>нечто</i> (кавычки необязательны)
<code>RSiteSearch("нечто")</code>	Поиск записей, содержащих <i>нечто</i> в онлайн-руководствах и заархивированных рассылках
<code>apropos("нечто", mode="function")</code>	Список всех доступных функций, в названии которых есть <i>нечто</i>
<code>data()</code>	Список всех демонстрационных данных, содержащихся в загруженных пакетах
<code>vignette()</code>	Список всех доступных руководств по загруженным пакетам
<code>vignette("нечто")</code>	Список руководств по теме <i>нечто</i>

Пояснения к пп. 1-4.

На рисунке 1 приведен интерфейс RStudio: окно сценария, окно окружения, окно консоли, окно диаграмм.

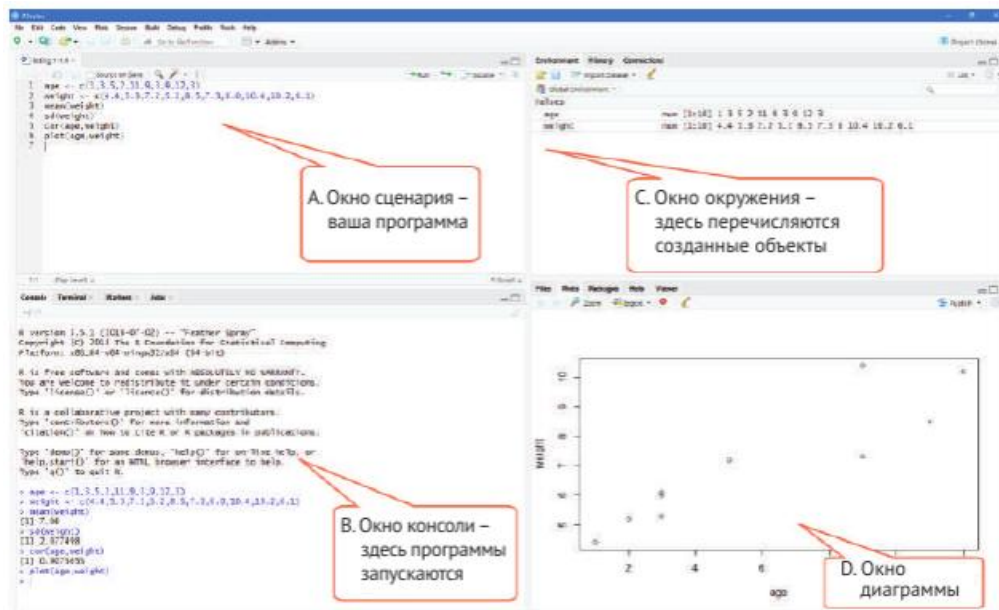


Рисунок 1 – Интерфейс RStudio

Окно сценария

В меню File (Файл) выберите пункт New File > R Script (Новый файл > Сценарий R).

В левом верхнем углу откроется новое окно сценария. Введите в него код, для примера:

```
x<-c(2,3,10,15,12, 1,4,8)
y<-c(5,7,19,28,25,2,10,15)
plot(x,y)
```

По мере ввода редактор будет подсвечивать синтаксис и предлагать варианты завершения кода. Например, по мере ввода `plot` появится всплывающее окно с именами доступных функций, начинающимися с букв, набранных к текущему моменту. Вы можете использовать клавиши со стрелками вверх и вниз, чтобы выбрать функцию из списка, и нажать Tab, для ее выбора. В круглых скобках, следующих за именем функции, можно нажать Tab, чтобы просмотреть список параметров функции.

Чтобы выполнить код, выделите его и щелкните на кнопке Run (Выполнить) или нажмите Ctrl+Enter. Комбинация Ctrl+Shift+Enter запустит весь код, а не только выделенный код. Чтобы сохранить сценарий, щелкните на значке Save (Сохранить) или выберите пункт меню File > Save (Файл > Сохранить).

В открывшемся диалоге выберите имя для сценария и папку, куда его следует сохранить. По соглашению файлы сценариев получают расширение .R. Имя файла сценария выводится на вкладке окна с красной звездочкой, если текущая версия не была сохранена.

В результате выполнения кода будет создано два вектора: `x` и `y`, построена диаграмма рассеяния (рисунок 2).

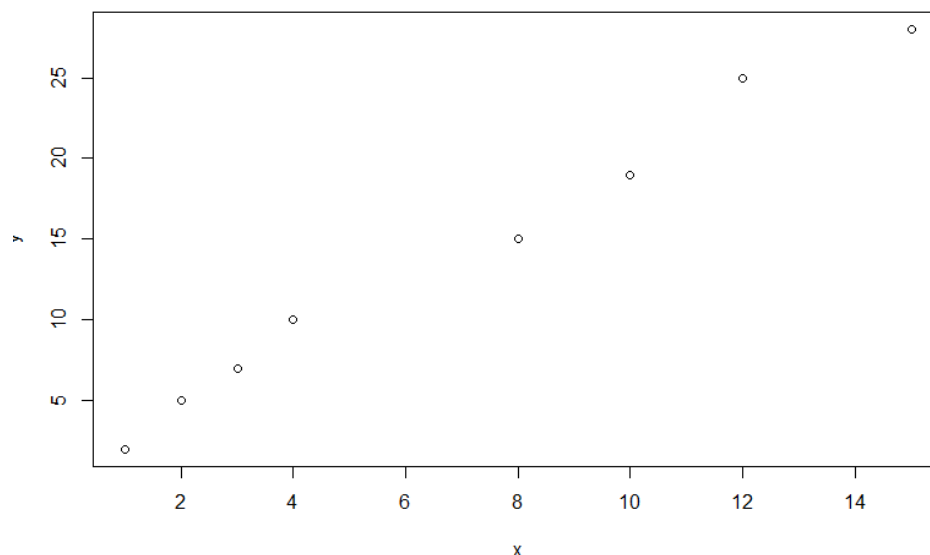


Рисунок 2 – Диаграмма рассеяния

Окно консоли

Код запускается в окне консоли. По сути, это та же консоль, которую можно видеть в базовом интерфейсе R. Код из окна сценария в окно консоли можно отправлять с помощью команды `Run` (Выполнить) или вводить команды непосредственно в этом окне, в строке приглашения к вводу (`>`). Если приглашение к вводу меняется на знак плюс (`+`), то это означает, что интерпретатор ожидает завершения инструкции. Такое часто происходит, когда инструкция слишком длинная и не умещается на одной строке или если в коде есть непарные круглые скобки. Вы можете отменить ввод инструкции и вернуться в командную строку, нажав клавишу `Esc`. Кроме того, клавиши со стрелками вверх и вниз будут циклически переключать прошлые выполненные инструкции. Вы можете отредактировать инструкцию и повторно запустить ее клавишей `Enter`. Щелчок на значке с изображением метлы удаляет текст из окна.

Окно окружения и истории

Любые создаваемые объекты (в данном примере `x` и `y`) будут появляться в окне окружения, а история выполненных команд будет сохраняться в окне истории – вкладка `History` (История) правее вкладки `Environment` (Окружение).

Окно диаграммы

Любые диаграммы, созданные сценарием, появятся в окне диаграммы. Панель инструментов этого окна позволяет циклически перемещаться по созданным диаграммам. Кроме того, окно диаграммы можно масштабировать, чтобы рассматривать диаграммы в разных масштабах, экспортировать диаграммы в несколько форматов и удалить одну или все диаграммы, созданные к настоящему моменту.

Рабочее пространство – это текущая рабочая среда R в памяти вашего компьютера, которая включает в себя любые созданные пользователем объ-

екты (векторы, матрицы, функции, таблицы данных или списки). В конце каждой сессии вы можете сохранить рабочее пространство, и оно автоматически загрузится при следующем запуске программы. Команды интерактивно вводятся в ответ на приглашение к их вводу. Можно использовать стрелки вверх и вниз для перемещения между введенными ранее командами. Это позволяет вызвать предыдущую команду, отредактировать ее и вновь выполнить ее, нажав клавишу **Enter**. Текущая рабочая директория – это та директория, где находятся файлы данных и куда по умолчанию сохраняются результаты. Функция **getwd()** позволяет узнать, какая директория в данный момент является рабочей.

Вы можете назначить рабочую директорию при помощи функции:
`setwd("C:\\project1")`

Работа в R начинается, как правило, с установки рабочей директории.

Если появляется необходимость импортировать файл, который находится не в рабочей директории, нужно написать полный путь к нему. Всегда заключайте в кавычки названия файлов и директорий. Некоторые стандартные команды для управления рабочим пространством приведены в таблице 2.

Таблица 2. – Функции управления рабочим пространством

Функция	Действие
<code>getwd()</code>	Вывести на экран название текущей рабочей директории
<code>setwd("моя_директория")</code>	Назначить <i>моя_директория</i> текущей рабочей директорией
<code>ls()</code>	Вывести на экран список объектов в текущем рабочем пространстве
<code>rm("список_объектов")</code>	Удалить один или несколько объектов
<code>help(options)</code>	Справка о возможных опциях
<code>options()</code>	Посмотреть или установить текущие опции
<code>history(#)</code>	Вывести на экран последние # команд (по умолчанию 25)
<code>savehistory("мой_файл")</code>	Сохранить историю команд в файл <i>мой_файл</i> (по умолчанию <i>.Rhistory</i>)
<code>loadhistory("мой_файл")</code>	Загрузить историю команд (по умолчанию <i>.Rhistory</i>)
<code>save.image("мой_файл")</code>	Сохранить рабочее пространство в файл <i>мой_файл</i> (по умолчанию <i>.Rdata</i>)

<code>save("список_объектов", file="мой_файл")</code>	Сохранить определенные объекты в файл
<code>load("мой_файл")</code>	Загрузить сохраненное рабочее пространство в текущую сессию (по умолчанию .Rdata)
<code>q()</code>	Выйти из программы. Появится вопрос, нужно ли сохранить рабочее пространство

Таблицы данных – это основной класс объектов R, используемых для хранения данных. Обычно такие таблицы подготавливаются при помощи сторонних приложений (особенно популярна и удобна программа Microsoft Excel) и затем загружаются в среду R (https://r-analytics.blogspot.com/2011/07/r_22.html).

Для загрузки данных из CSV-файла с заголовками используется команда:

```
data <- read.table("C:\\Users\\Пользователь\\Desktop\\MAD\\var_1.csv", header=TRUE, sep=";")
```

read.table - это функция (процедура).

В круглых скобках - аргументы функции.

("C:\\Users\\Пользователь\\Desktop\\вариант-1(л.р.1).csv" - путь к файлу с данными

header= указываем, есть в первой строке названия переменных, или нет

sep= указываем, какой символ отделяет поля

row.names= столбец, в котором хранятся названия строк (параметр не обязателен).

В результате выполнения команды создается объект таблица с названием data.

Возможно, что в загруженном файле некорректно будет отображаться текст на русском языке из-за установленных кодировок. В этом случае необходимо использовать команду:

```
Sys.setlocale("LC_ALL", "Russian")
```

Файл можно также предварительно открыть в текстовом редакторе и сохранить в другой кодировке, либо явно задать аргумент fileEncoding или encoding в функции read.table(), например: fileEncoding = "Windows-1251", encoding = "UTF-8", или encoding = "1251". Выбор параметра зависит от используемой операционной системы!

Для записи данных в CSV-файл используется команда:

```
write.table(data, file = "C:\\Users\\ Пользователь \\Desktop\\MAD\\var_1.csv", sep = ";", row.names = TRUE, col.names = NA)
```

Для импорта/экспорта данных из rds команды:

```
saveRDS(data, "data.rds")
```

```
data <- readRDS("data.rds")
```

Для импорта/экспорта данных из RData команды:

```
save(data, cdata, file="data.RData")
```

```
load("data.RData")
```

Для просмотра таблицы с данными используется команда:

```
View(data)
```

Для обращения к конкретному столбцу таблицы используется команда:

```
data$column_name
```

Для обращения к элементу столбца (например, к строке три) используется команда:

```
data$column_name[3]
```

Обратите внимание! Нумерация строк и столбцов в таблице начинается с единицы!

Для обращения к строке 1, в столбце 5 используется команда:

```
data[1,5]
```

Можно делать различные выборки из данных.

Например, вывод данных из столбца column_name_1, для которых значение в столбце column_name_2 больше 70:

```
View(data$column_name_1[data$column_name_2 > 70])
```

Также можно использовать команду subset для формирования подвыборки из БД:

```
View(subset(data, column_name_2 > 70))
```

Можно выбирать одновременно строки и столбцы и сформировать новую БД:

```
data1 <- subset(data, column_name_2 > 70, select=c("column_name_1", "column_name_3"))
```

Вывод первых трех строчек столбца column_name:

```
View(data$column_name[1:3])
```

Вывод первых трех столбцов данных:

```
View(data[1:3])
```

Вывод первого, третьего и пятого столбцов данных:

```
View(data[c(1,3,5)])
```

Вывод первых 100 строк данных:

```
View(data[1:100,])
```

Удаление записей, содержащих пропущенные значения:

```
data <- na.omit(data)
```

Удаление колонки:

```
data$column_name <- NULL
```

или

```
data <- data[, -c(1:10)]
```

В результате будут удалены первые 10 столбцов с данными.

Удаление строк:

```
data <- data[-c(1:99), ]
```

В результате будут удалены первые 99 строк с данными.

Вызов редактора данных:

```
data <- edit(data)
```

Если в таблице с исходными данными длинные названия столбцов, то лучше ввести сокращенные названия для качественного визуального представления графиков и результатов анализа!

Для переименования нескольких столбцов:

```
names(data) <- c("возраст", "стаж", "часы", "доход")
```

Для переименования одного столбца, в квадратных скобках номер столбца:

```
colnames(data)[6] <- "процент_успешных_задач"
```

```
colnames(data)[7] <- "количество_ошибок"
```

```
colnames(data)[8] <- "удовлетворенность_б"
```

```
colnames(data)[9] <- "удовлетворенность_к"
```

```
colnames(data)[10] <- "документирование"
```

В R реализован ряд функций, которые позволяют определить тип данных и преобразовать их в другой формат. Эти преобразования происходят в R сходным с другими языками программирования образом. Например, если добавить текстовые данные к числовому вектору, все значения вектора преобразуются в текстовый формат. Для проверки типа данных и преобразования их в другой формат можно использовать функции, перечисленные в таблице 3.

Таблица 3. – Функции, используемые для изменения формата данных

Проверка	Преобразование
<code>is.numeric()</code>	<code>as.numeric()</code>
<code>is.character()</code>	<code>as.character()</code>
<code>is.vector()</code>	<code>as.vector()</code>
<code>is.matrix()</code>	<code>as.matrix()</code>
<code>is.data.frame()</code>	<code>as.data.frame()</code>
<code>is.factor()</code>	<code>as.factor()</code>
<code>is.logical()</code>	<code>as.logical()</code>

Функции типа `is._()` возвращают TRUE или FALSE, тогда как функции типа `as._()` преобразуют данные в соответствующий формат. Пример представлен в приведенном ниже программном коде.

```
a <- c(1,2,3)
a
[1] 1 2 3
is.numeric(a)
[1] TRUE
is.vector(a)
[1] TRUE
a <- as.character(a)
a
```

При работе с большими таблицами данных бывает сложно визуально исследовать всё их содержимое перед началом анализа. Однако визуального просмотра содержимого таблиц и не требуется – полную сводную информацию о них (равно как и о других объектах R) можно легко получить при помощи команды `str()` (structure – структура):

```
str(data)
```

В результате:

```
'data.frame': 200 obs. of 10 variables:
 $ N      : int  1 2 3 4 5 6 7 8 9 10 ...
 $ Group   : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Gender   : int  1 2 2 2 1 1 2 2 2 2 ...
 $ Age      : int  31 27 31 28 30 26 28 31 28 30 ...
 $ Experience : num  2.3 2.8 3.2 3.5 2.9 3 3.7 4.1 2.5 3.3 ...
 $ x..of.developments.completed.on.time : int  83 90 83 84 89 85 91 94 87 90 ...
 $ Number.of.errors.identified.by.the.user: int  24 18 28 18 15 21 20 24 22 19 ...
 $ Customer.satisfaction..score. : int  61 73 62 68 80 70 86 78 72 85 ...
 $ Customer.satisfaction..level. : chr   "Low" "Medium" "Low" "Medium" ...
 $ Quality.of.documentation : int  2 2 1 1 2 2 4 4 3 3 ...
```

Для изменения типа признака Gender (пол) в таблице data:

```
data$Gender<-as.factor(data$Gender)
```

str(data)

В результате:

```
'data.frame': 200 obs. of 10 variables:
 $ N                : int  1 2 3 4 5 6 7 8 9 10 ...
 $ Group            : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Gender           : Factor w/ 2 levels "1","2": 1 2 2 2 1 1 2 2 2 2 ...
 $ Age             : int  31 27 31 28 30 26 28 31 28 30 ...
 $ Experience       : num  2.3 2.8 3.2 3.5 2.9 3 3.7 4.1 2.5 3.3 ...
 $ X.of.developments.completed.on.time : int  83 90 83 84 89 85 91 94 87 90 ...
 $ Number.of.errors.identified.by.the.user: int  24 18 28 18 15 21 20 24 22 19 ...
 $ Customer.satisfaction.score.         : int  61 73 62 68 80 70 86 78 72 85 ...
 $ Customer.satisfaction.level.         : chr  "Low" "Medium" "Low" "Medium" ...
 $ Quality.of.documentation             : int  2 2 1 1 2 2 4 4 3 3 ...
```

Пояснения к п. 5.

Для расчета основных статистических характеристик по данным используется команда:

summary(data)

В результате выполнения команды summary(data) для факторов (качественных признаков) выводится информация об уровнях, которые может принимать фактор и количестве наблюдений на каждом уровне, для количественных переменных выводятся значения основных числовых характеристик: минимальное и максимальное значения, среднее, медиана, первый и третий квартили.

Пример вывода:

Gender	Age	Experience
1: 75	Min. :24.00	Min. :1.500
2:125	1st Qu.:29.00	1st Qu.:3.175
	Median :31.00	Median :4.350
	Mean :31.08	Mean :5.038
	3rd Qu.:33.00	3rd Qu.:6.950
	Max. :39.00	Max. :8.900

Также для вычисления статистических характеристик могут использоваться статистические функции, представленные в таблице 4.

Таблица 4. – Статистические функции

Функция	Описание
<code>mean(x)</code>	Среднее арифметическое <code>mean(c(1, 2, 3, 4))</code> равно 2.5
<code>median(x)</code>	Медиана <code>median(c(1, 2, 3, 4))</code> равно 2.5
<code>sd(x)</code>	Стандартное отклонение <code>sd(c(1, 2, 3, 4))</code> равно 1.29
<code>var(x)</code>	Дисперсия <code>var(c(1, 2, 3, 4))</code> равно 1.67
<code>mad(x)</code>	Абсолютное отклонение медианы <code>mad(c(1, 2, 3, 4))</code> равно 1.48
<code>quantile(x, probs)</code>	Квантили, где <i>x</i> – числовой вектор, для которого нужно вычислить квантили, а <i>probs</i> – числовой вектор с указанием вероятностей в диапазоне [0; 1] # 30-й и 84-й процентиля <i>x</i> <code>y <- quantile(x, c(.3, .84))</code>
<code>range(x)</code>	Размах значений <code>x <- c(1, 2, 3, 4)</code> <code>range(x)</code> равно <code>c(1, 4)</code> . <code>diff(range(x))</code> равно 3
<code>sum(x)</code>	Сумма <code>sum(c(1, 2, 3, 4))</code> равно 10
<code>diff(x, lag=n)</code>	Разность значений в выборке, взятых с заданным интервалом (<i>lag</i>). По умолчанию интервал равен 1. <code>x <- c(1, 5, 23, 29)</code> <code>diff(x)</code> равно <code>c(4, 18, 6)</code>
<code>min(x)</code>	Минимум <code>min(c(1, 2, 3, 4))</code> равно 1
<code>max(x)</code>	Максимум <code>max(c(1, 2, 3, 4))</code> равно 4
<code>scale(x, center=TRUE, scale=TRUE)</code>	Значения объекта <i>x</i> , центрованные (<i>center=TRUE</i>) или стандартизованные (<i>center=TRUE, scale=TRUE</i>) по столбцам. Пример дан в программном коде 5.6

В базовой версии R не вычисляются статистические характеристики асимметрии и эксцесса, но можно написать собственную функцию для вычисления этих характеристик или установить пакет, в котором реализовано вычисление характеристик.

В R пользователи могут создавать свои функции. Структура этих функций выглядит примерно так:

```
myfunction <- function(arg1, arg2, ... )
{ statements
return(object) }
```

Объекты внутри функции не доступны вне функции. Объект, который возвращается в результате действия функции, может быть любого типа – от скаляра до списка.

Функция для вычисления статистических характеристик выглядит следующим образом:

```
mystats <- function(x)
{
  if(is.numeric(x))
  { m <- mean(x)
    n <- length(x)
    s <- sd(x)
    skew <- sum((x-m)^3/s^3)/n
    kurt <- sum((x-m)^4/s^4)/n - 3

    return(c(n=n, mean=m, stdev=s, skew=skew, kurtosis=kurt))
  }
}
```

Вызов функции для вычисления статистических характеристик столбца name_1:

```
mystats(data$name_1)
```

Многие R-пакеты имеют собственные функции, аналогичные стандартной summary(), для вывода компактных описательных сводок по таблицам данных. Например, пакет psych имеет функцию describe().

```
install.packages("psych")
library(psych)
sm<-describe(data[,c(4,5)])
View(sm)
```

В результате выполнения кода значения статистических характеристик для четвертого и пятого столбцов data будут выведены в таблицу:

	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew	kurtosis	se
Age	1	200	31.0800	2.911354	31.00	31.05625	2.96520	24.0	39.0	15.0	0.08045784	-0.1239651	0.2058638
Experience	2	200	5.0385	2.091204	4.35	4.98375	2.59455	1.5	8.9	7.4	0.16624141	-1.5591596	0.1478704

Пояснения к п. 6.

В R реализованы обширные возможности для работы с графикой.

Например, для построения диаграммы рассеяния используется команда:

```
plot(data.frame(data$name_1,data$name_2))
```

#name_1 – имя признака, значения которого откладываются по оси X

#name_2 – имя признака, значения которого откладываются по оси Y.

У plot можно задать большое количество дополнительных параметров, список которых можно посмотреть, вызвав help(plot).

Например,

```
plot(data.frame(data$name_1,data$name_2), type="b", col="red", lty=2, pch=2,
lwd=2, main="название_графика", sub="подпись_графика",
xlab="название_оси_X", ylab="название оси Y", xlim=c(0, 60), ylim=c(0,
70))
```

#type= - тип графика

#col= - цвет элементов

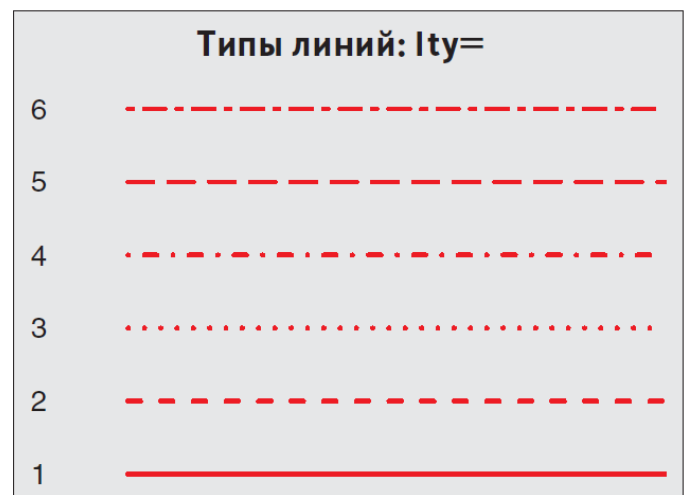
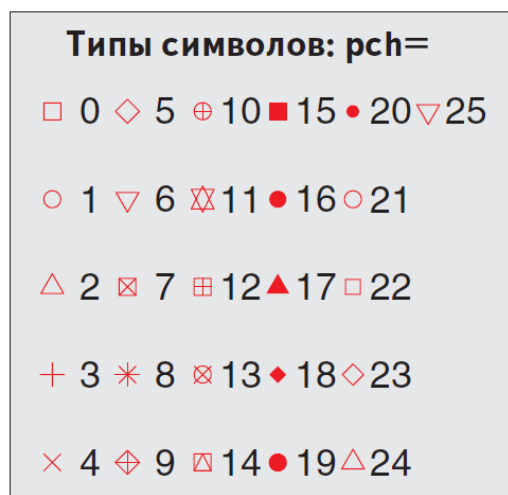
#lty= - тип линии (2 – dashed, пунктирный)

#pch= - тип точки

#lwd= - ширина линии

#xlim= - диапазон изменения оси X

#ylim= - диапазон изменения оси Y



Пример:

```
plot(data.frame(data$Age, data$Experience), type="p", col = "red", lty=2,
pch=19, lwd=2, main="Диаграмма рассеяния", sub="Зависимость между воз-
растом и стажем", xlab="возраст", ylab="стаж", xlim=c(22, 41), ylim=c(0, 11))
```

В результате график:

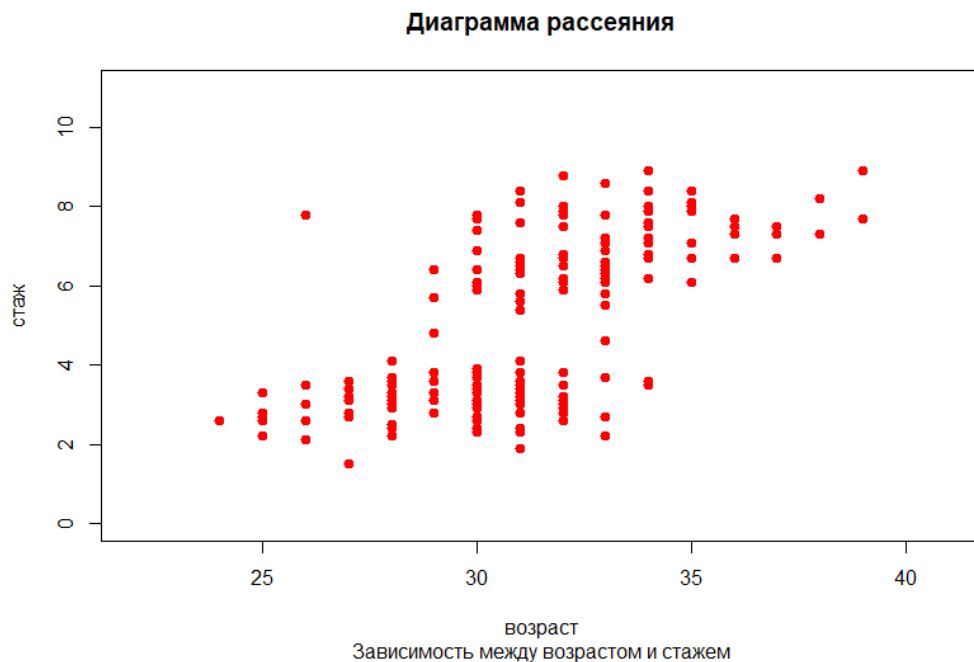


Рисунок 3 – Пример диаграммы рассеяния

В стандартной интерактивной сессии можно создавать график, вводя по одной команде и добавляя элементы графика, пока не получите то, что хотели.

Например:

```
attach(data)
plot(data$name_1, data$name_2)
abline(lm(name_2~name_1))
title("name")
detach(data)
```

Первая команда добавляет в траекторию поиска таблицу данных `data`. Вторая команда открывает окно графики и создает диаграмму рассеяния. Третья команда добавляет регрессионную прямую. Четвертая команда добавляет название. Последняя команда удаляет таблицу данных из пути поиска.

Диаграммы можно сохранять при помощи программного кода или меню графического пользовательского интерфейса. Для сохранения диаграммы при помощи кода разместите создающие диаграмму команды между командами, которые назначают место вывода и закрывают вывод. Например, следующий программный код позволяет сохранить диаграмму в формате PNG под названием `mygraph.png` в текущей рабочей директории:

```
png("mygraph.png")
attach(data)
plot(data$name_1, data$name_2)
abline(lm(name_2~name_1))
title("name")
```

dev.off()

Можно сохранить диаграмму при помощи графического пользовательского интерфейса. При активированном графическом окне нужно выбрать в меню **Файл** → **Сохранить как**, а затем в появившемся диалоге выбрать нужный формат графического файла и директорию для сохранения.

Новая диаграмма, которая создается при помощи команды высокого уровня, такой как `plot()`, `hist()` или `boxplot()`, заменяет предыдущую диаграмму. Как же создать более одной диаграммы и иметь доступ к каждой из них? Есть несколько способов. Во-первых, можно открыть новое графическое устройство, *перед* тем как создавать новую диаграмму:

`dev.new()`

Каждая новая диаграмма будет появляться в последнем открытом окне.

Во-вторых, можно получить доступ к нескольким диаграммам сразу через пользовательский интерфейс. После того как открыто *первое* окно графики, выберите в меню **История команд** → **Запись**. Затем используйте пункты меню **Предыдущий** и **Следующий** для перемещения между созданными диаграммами.

В-третьих, можно использовать функции `dev.new()`, `dev.next()`, `dev.prev()`, `dev.set()` и `dev.off()` для одновременного открытия нескольких окон графики и выбора необходимой диаграммы. Эти команды работают под всеми операционными системами. Введите `help(dev.cur)`, чтобы узнать больше об этом подходе.

Для построения радиальных диаграмм можно использовать команду `pie()`:

```
x<-c(summary(data$name_1))
piepercent<- round(100*x/sum(x), 1)
pie(x, piepercent, radius=1, main="Радиальная диаграмма", xlab="название оси X", ylab="название оси Y", col=c("red", "blue"), clockwise=TRUE)
legend("topright", c("Yes","No"), cex = 0.8, fill =c("red", "blue"))
```

В данном примере признак `name_1` является факторным (качественным) и принимает два значения (1 и 0). Создается объект `x`, который хранит количество 0 и 1 и объект `piepercent` – количество в процентах. Далее строится радиальная диаграмма, состоящая из красного и синего секторов с указанием процентов. Последняя команда задает легенду, расположенную в правом верхнем углу графика, параметр `cex` задает размер окна легенды.

Пример:

```
x<-c(summary(data$Quality.of.documentation))
piepercent<- round(100*x/sum(x), 1)
piepercent
```

```
pie(x, piepercent, radius=1, main="Радиальная диаграмма",
xlab="Качество документирования", col=c("red", "blue", "green", "orange"),
clockwise=TRUE)
```

```
legend("topright", c("1","2","3", "4"), cex = 0.8, fill =c("red", "blue",
"green", "orange"))
```

В результате:



Рисунок 4 – Пример радиальной диаграммы

Также можно размещать в одном графическом пространстве несколько диаграмм. Например, построим две столбиковые диаграммы по признаку name_2: первая диаграмма построена по значениям, для которых признак name_1="Yes"; вторая построена по значениям, для которых признак name_1="No":

```
par(mfrow = c(1, 2))
barplot(table(data$name_2[data$name_1=="Yes"]), col=c("red"), main="Yes" ,
xlab="название оси X", ylab="название оси Y")
barplot(table(data$name_2[data$name_1=="No"]), col=c("blue"), main="No",
xlab="название оси X", ylab="название оси Y")
```

Первая команда задает разбиение графического пространства (сколько графиков будет по строке и по столбцу). Вторая и третья команды строят столбиковую диаграмму в зависимости от условий.

Пример:

```
par(mfrow = c(1, 2))
barplot(table(data$Experience[data$Gender=="1"]), col=c("red"),
main="Мужчины", xlab="стаж работы", ylab="количество наблюдений")
barplot(table(data$Experience[data$Gender=="2"]), col=c("blue"),
main="Женщины", xlab="стаж работы", ylab="количество наблюдений")
```

В результате:

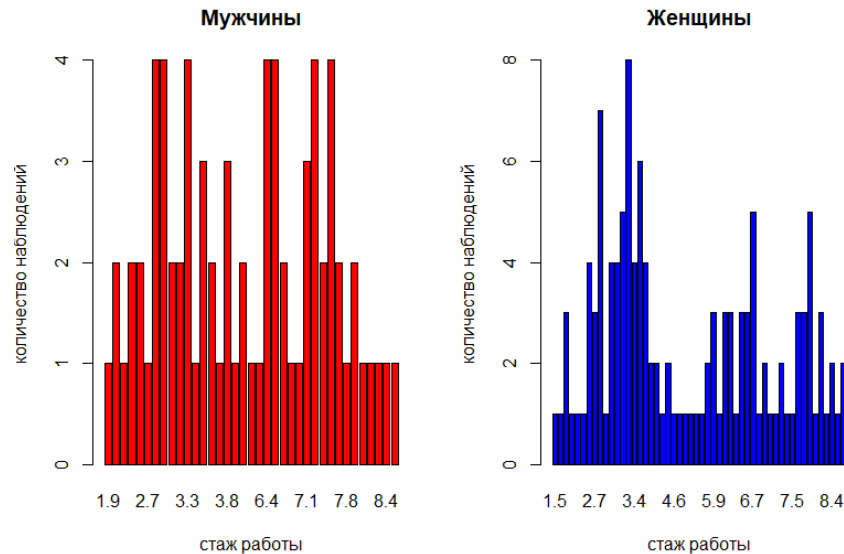


Рисунок 5 – Пример категориальной столбиковой диаграммы по полу

Пример:

```
par(mfrow = c(2, 2))
barplot(table(data$Experience[(data$Gender=="1")&(data$Group=="1")]),
col=c("red"), main="Мужчины первой группы", xlab="стаж работы",
ylab="количество наблюдений")
barplot(table(data$Experience[(data$Gender=="1")&(data$Group=="2")]),
col=c("green"), main="Мужчины второй группы", xlab="стаж работы",
ylab="количество наблюдений")
barplot(table(data$Experience[(data$Gender=="2")&(data$Group=="1")]),
col=c("blue"), main="Женщины первой группы", xlab="стаж работы",
ylab="количество наблюдений" )
barplot(table(data$Experience[(data$Gender=="2")&(data$Group=="2")]),
col=c("orange"), main="Женщины второй группы", xlab="стаж работы",
ylab="количество наблюдений" )
```

В результате:

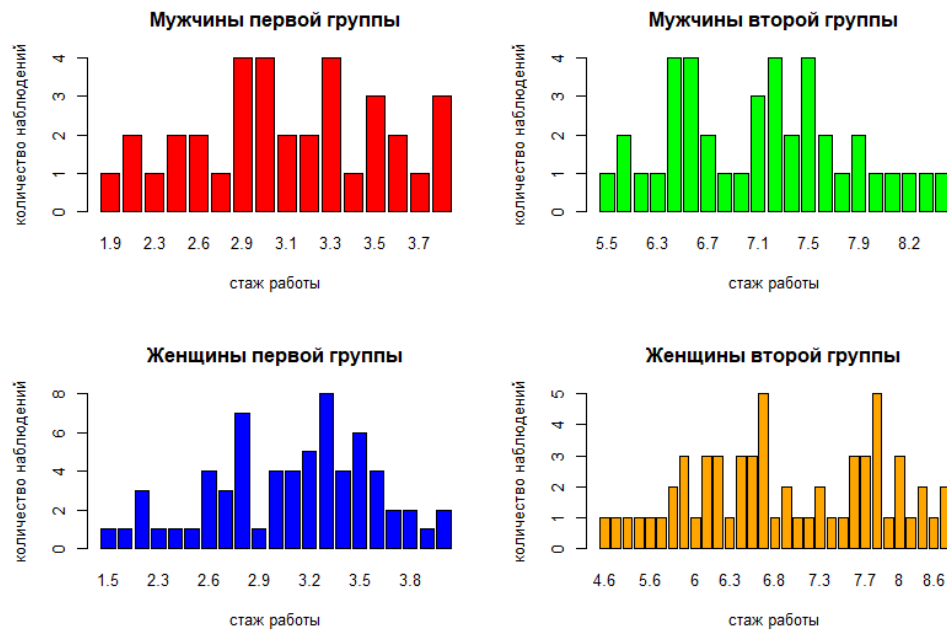


Рисунок 6 – Пример категориальной столбиковой диаграммы по полу и группе

Диаграммы размахов (box plot) иллюстрируют распределение значений количественной переменной, отображая пять параметров: минимум, нижний квартиль (75-й процентиль), медиану (50-й процентиль), верхний квартиль (25-й процентиль) и максимум. На этой диаграмме также могут быть отображены вероятные выбросы (значения, выходящие за диапазон в ± 1.5 межквартильного размаха, разности верхнего и нижнего квартилей).

Например, команда:

```
boxplot(data$name_3, main="Ящик с усами", ylab="Название оси Y")
```

 позволяет построить диаграмму, показанную на рисунке 7 (подписи к элементам добавлены вручную).

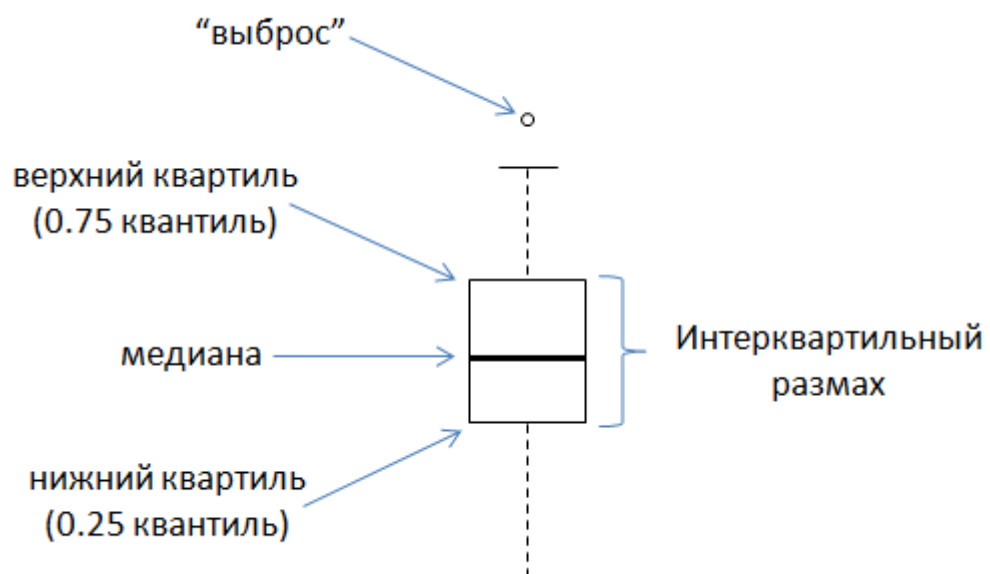


Рисунок 7 – Диаграмма размаха

По умолчанию каждый «ус» продолжается до минимального или максимального значения, которое не выходит за пределы 1.5 межквартильного размаха. Выходящие за эти пределы значения отмечаются точками. Диаграммы размахов можно построить для отдельных переменных или для групп переменных. Общий вид команды таков:

```
boxplot(formula, data=dataframe)
```

где *formula* – это формула, а *dataframe* обозначает таблицу данных (или список), где содержатся данные.

Примером формулы может служить выражение $y \sim A$, где для каждого значения качественной переменной *A* будет построена отдельная диаграмма размахов для количественной переменной *y*. Формула $y \sim A * B$ позволит получить отдельные диаграммы размахов для всех комбинаций значений переменной *y*, заданных категориальными переменными *A* и *B*. Параметр `varwidth=TRUE` позволяет получить диаграмму, на которой ширина «ящиков» будет пропорциональна квадратному корню из размера выборки. Используйте параметр `horizontal=TRUE`, чтобы поменять оси местами. Если добавить параметр `notch=TRUE`, то получатся «ящики» с «насечками». Если «насечки» двух ящиков не перекрываются, высока вероятность того, что медианы соответствующих совокупностей различаются (Chambers et al., 1983, стр. 62).

Например, команда:

```
par(mfrow=c(1,1))
boxplot(data$name_3 ~ data$name_1,
+       xlab = "Название оси X",
+       ylab = "Название оси Y",
+       col = "coral", data = data)
```

Переменная *name_3* – количественная, *name_1* – качественная, принимающая два значения (1 и 0). В результате выполнения команды будет построено два ящика с усами на одном графике.

Пример:

```
par(mfrow=c(1,1))
boxplot(data$Experience ~ data$Gender, xlab = "Пол", ylab = "Значения статисти-
ческих характеристик стажа работы", col = "coral", data = data)
```

В результате:

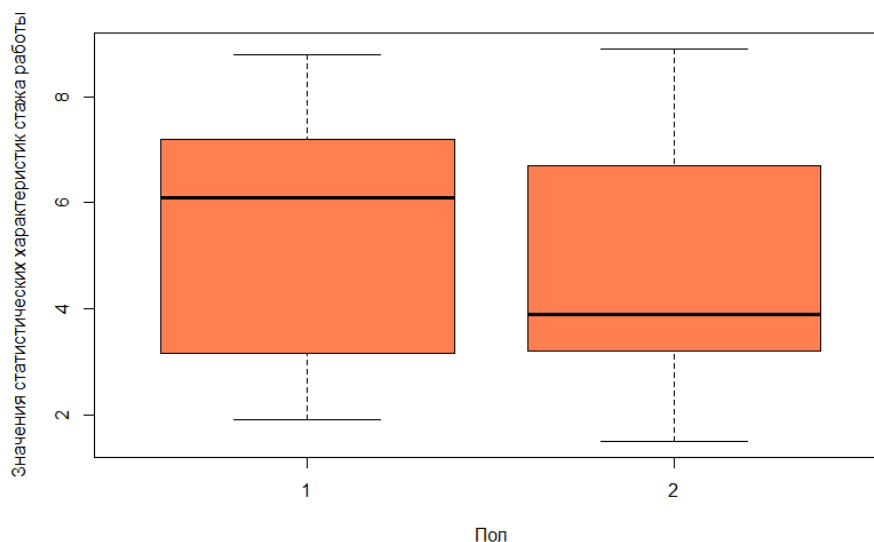


Рисунок 8 – Пример диаграммы размаха

Гистограмма (оценка функции плотности распределения вероятностей) графически отображает распределение значений количественных переменных, разделяя диапазон значений на заданное число отрезков по оси x и отображая частоту значений внутри каждого отрезка на оси y . Гистограмму можно создать при помощи функции:

`hist(x)`, где x – это числовой вектор.

Опция `freq=FALSE` позволяет построить гистограмму на основе плотности вероятности, а не частот значений. Параметр `breaks` определяет число отрезков. По умолчанию все отрезки имеют одинаковую длину.

Пример построения гистограммы с наложением кривой плотности распределения вероятностей:

```
hist(data$name_1, freq=FALSE, breaks=12, col="red", xlab="название оси X",
+ main="Гистограмма")
lines(density(data$name_1))
```

Пример:

```
par(mfrow = c(1,2))
hist(data$Age, xlab="возраст", ylab="Частота наблюдений", breaks=7,
col="red", freq=FALSE)
lines(density(data$Age))
hist(data$Experience, xlab="стаж", ylab="Частота наблюдений",
breaks=7, col="blue", freq=FALSE)
lines(density(data$Experience))
```

В результате:

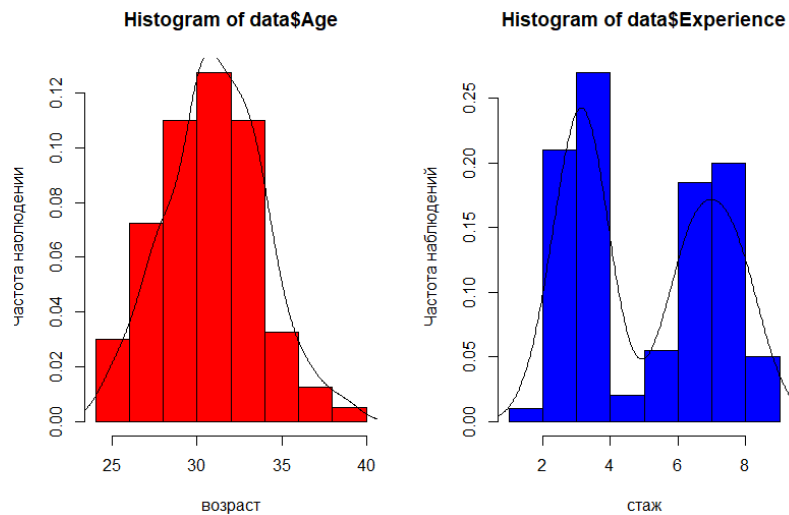


Рисунок 9 – Пример гистограмм

Построение гистограммы с наложением функции плотности нормального закона распределения:

```
install.packages("rcompanion")
library(rcompanion)
par(mfrow = c(1,2))
plotNormalHistogram(data$Age, xlab="возраст", ylab="Количество на-
блюдений", length = 1000, breaks = seq(min(data$Age), max(data$Age),
length.out = 7))
plotNormalHistogram(data$Experience, xlab="стаж", ylab="Количество
наблюдений", length = 1000, breaks = seq(min(data$Experience),
max(data$Experience), length.out = 7))
```

В результате:

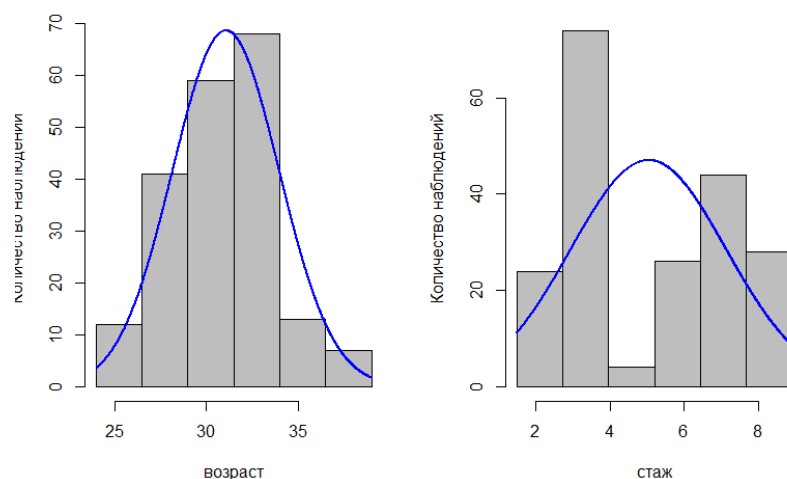


Рисунок 10 – Пример гистограмм с наложением функции НЗР

Матрицу диаграмм рассеяния можно построить с помощью функции:

```
pairs(~data$name_1+data$name_2+data$name_3, data=data, main="Матричный график")
```

Все переменные, перечисленные справа от знака ~, попадают на диаграмму. На этой диаграмме отражены взаимосвязи между всеми парами переменных. Например, диаграмма рассеяния для name_1 и name_2 расположена на пересечении строки и столбца с названиями этих переменных. Изменяя параметры команды, можно отображать только верхний или только нижний треугольник относительно главной диагонали. Например, опция `upper.panel=NULL` позволит оставить только нижний треугольник с диаграммами.

Пример:

```
pairs(~data$Age+data$Experience+data$Number_of_errors, data=data,
main="Матричный график", col="red")
```

В результате:

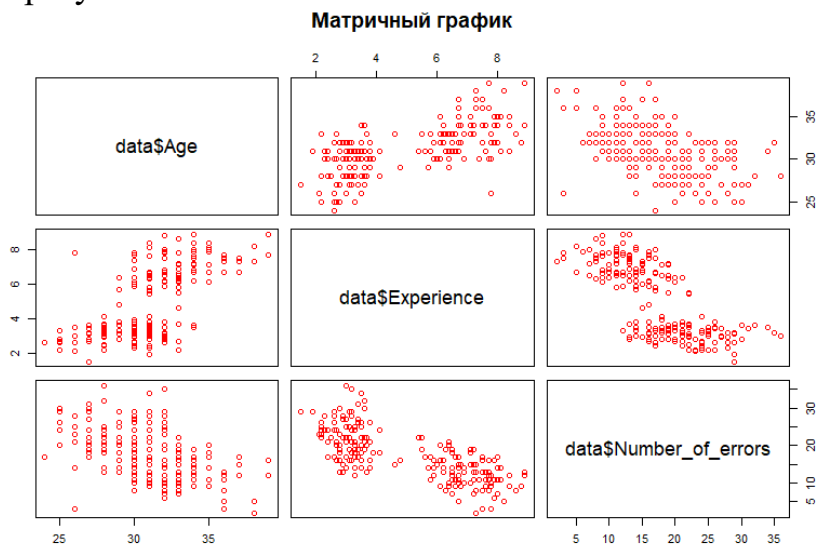


Рисунок 11 – Пример матричного графика

Пояснения к п 7.

Пояснения к п. 7.1.

Для аналитической проверки гипотезы о соответствии нормальному закону распределения могут быть применены критерии согласия: хи-квадрат, Шапиро – Уилка, Крамера – Мизеса, Андерсона – Дарлинга и другие. Функции, реализующие разные критерии согласия, представлены в пакете `nortest`:

```
install.packages("nortest")
library(nortest)
```

Пример расчета критерия согласия хи-квадрат для проверки гипотезы о нормальном законе распределения значений признака Experience – стаж в первой группе:

```
data1<-subset(data, data$Group=="1")  
pearson.test(data1$Experience)
```

В результате:

```
Pearson chi-square normality test  
  
data: data1$Experience  
P = 16.22, p-value = 0.09351
```

Пример расчета критерия согласия Андерсона-Дарлинга для проверки гипотезы о нормальном законе распределения значений признака Experience – стаж в первой группе:

```
ad.test(data1$Experience)
```

В результате:

```
Anderson-Darling normality test  
  
data: data1$Experience  
A = 0.71683, p-value = 0.05957
```

Пример расчета критерия согласия Шапиро-Уилка для проверки гипотезы о нормальном законе распределения значений признака Experience – стаж в первой группе:

```
shapiro.test(data1$Experience)
```

В результате:

```
shapiro-wilk normality test  
  
data: data1$Experience  
W = 0.97717, p-value = 0.07981
```

Пример расчета критерия согласия Крамера – Мизеса для проверки гипотезы о нормальном законе распределения значений признака Experience – стаж в первой группе:

```
cvm.test(data1$Experience)
```

В результате:

Cramer-von Mises normality test

```
data: data1$Experience  
w = 0.12015, p-value = 0.05862
```

Гипотеза о нормальном распределении значений признака «стаж» в первой группе не отвергается при уровне значимости 0,05 ($p\text{-value} > 0,05$) по всем критериям согласия.

Построение гистограммы с наложением функции плотности нормального закона распределения:

```
install.packages("rcompanion")  
library(rcompanion)  
par(mfrow = c(1,2))  
plotNormalHistogram(data$Age, xlab="возраст", ylab="Количество на-  
блюдений", length = 1000, breaks = seq(min(data$Age), max(data$Age),  
length.out = 7))  
plotNormalHistogram(data$Experience, xlab="стаж", ylab="Количество  
наблюдений", length = 1000, breaks = seq(min(data$Experience),  
max(data$Experience), length.out = 7))
```

В результате:

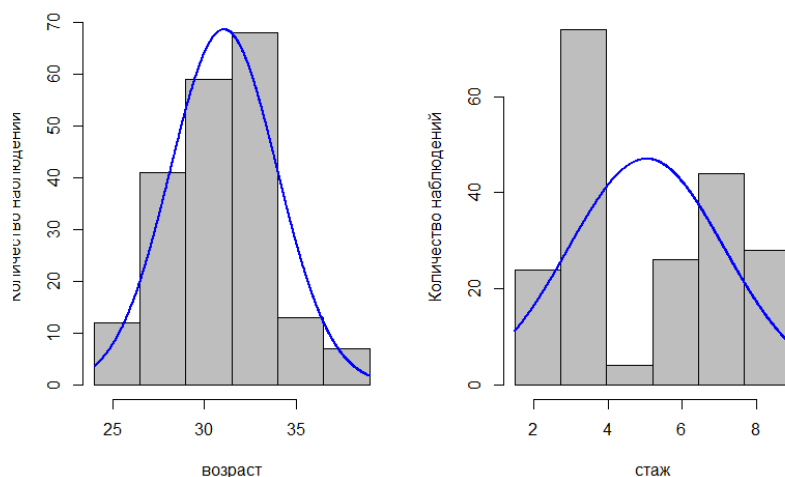


Рисунок 12– Пример гистограмм с наложением функции НЗР

Пояснения к п. 7.2.

В R можно рассчитывать разные коэффициенты корреляции для оценки взаимосвязи между количественными и порядковыми признаками, включая коэффициенты Пирсона, Спирмена, Кендалла, частные, полихорические и многорядные.

Линейный коэффициент корреляции Пирсона (Pearson product moment correlation) отражает степень линейной связи между двумя количественными признаками. Коэффициент ранговой корреляции Спирмена (Spearman's Rank Order correlation) – мера связи между двумя ранжированными признаками. Тау Кендалла (Kendall's Tau) – также непараметрический коэффициент ранговой корреляции.

Функция `cor()` позволяет вычислить все три коэффициента, а функция `cov()` рассчитывает ковариации. У этих функций есть много параметров, но упрощенный формат вычисления корреляций таков:

`cor(x, use= , method= )`.

Эти параметры описаны в таблице 1. Значения опций по умолчанию: `use="everything"` и `method="pearson"`.

Таблица 5 – Параметры (опции) функций `cor()` и `cov()`

Опция	Описание
<code>x</code>	Матрица или таблица данных
<code>use=</code>	Упрощает работу с пропущенными данными. Может принимать следующие значения: <code>all.obs</code> (предполагается, что пропущенные значения отсутствуют; их наличие вызовет сообщение об ошибке), <code>everything</code> (любая корреляция, включающая строку с пропущенным значением, не будет вычисляться – обозначается как <code>missing</code>), <code>complete.obs</code> (учитываются только строки без пропущенных значений) и <code>pairwise.complete.obs</code> (учитываются все полные наблюдения для каждой пары переменных в отдельности)
<code>method=</code>	Определяет тип коэффициента корреляции. Возможные значения – <code>pearson</code> , <code>spearman</code> или <code>kendall</code>

Ниже приведен пример вычисления коэффициентов корреляции между количественными признаками, хранящимися в таблице `data1`.

```
M <- data1[,unlist(lapply(data1, is.numeric))]
N1<-cor(M, use="pairwise.complete.obs")
N2<-cor(M, use="pairwise.complete.obs",method="spearman")
N3<-cor(M, use="pairwise.complete.obs",method="kendall")
View(N1)
View(N2)
View(N3)
```

Первая команда выбирает из таблицы data1 количественные признаки и сохраняет их в таблице M. Вторая команда вычисляет матрицу коэффициентов корреляции Пирсона между всеми количественными признаками, третья команда – матрицу коэффициентов корреляции Спирмена, четвертая команда – матрицу коэффициентов корреляции Кендалла.

Пояснения к п. 7.3.

Оценка значимости коэффициента корреляции Пирсона реализуется в функции cor.test(), conf.level – уровень доверительной вероятности, по умолчанию – 0.95.

Пример:

```
cor.test(data1$Experience,data1$Age, conf.level = 0.9)
```

В результате:

```
Pearson's product-moment correlation

data: data1$Experience and data1$Age
t = 2.0601, df = 98, p-value = 0.04204
alternative hypothesis: true correlation is not equal to 0
90 percent confidence interval:
 0.03959907 0.35717056
sample estimates:
      cor
0.203738
```

В результате выполнения функции выдается расчетное значение критической статистики (t), число степеней свободы (df), p-значение (p-value) и 90% доверительный интервал, в который попадает истинное значение коэффициента корреляции Пирсона.

Оценка значимости коэффициентов корреляции Спирмена и Кендалла реализуется в функции cor.test() при указании параметра method.

Пример:

```
cor.test(data1$Experience,data1$Age, method="spearman")
cor.test(data1$Experience,data1$Age, method="kendall")
```

Результат:

```

      kendall's rank correlation tau

data:  data1$Experience and data1$Age
z = 1.7807, p-value = 0.07497
alternative hypothesis: true tau is not equal to 0
sample estimates:
      tau 
0.1311596

```

Пояснения к п. 7.4.

Для вычисления коэффициентов частной корреляции Пирсона можно использовать функцию `pcor()` из пакета `ppcor`.

```

install.packages("ppcor")
library(ppcor)

```

Пример вычисления матрицы частных коэффициентов корреляции Пирсона для признаков 4-8 таблицы `data1`:

```

M1 <- data1[c(4:8)]
C1<-pcor(M1)

```

Вывод матрицы коэффициентов корреляции и р-значения для оценки статистической значимости коэффициента:

```

View(C1$estimate)
View(C1$p.value)

```

Матрицу частных коэффициентов корреляции также можно рассчитать на основе коэффициентов Спирмена и Кендалла:

```

C2<-pcor(M1, method="spearman")
C3<-pcor(M1, method="kendall")

```

Пояснения к п. 7.5.

Для графического представления степени взаимосвязей между признаками необходимо установить пакет `corrplot`:

```

install.packages("corrplot")
library(corrplot)

```

```

col <- colorRampPalette(c("#BB4444", "#EE9988", "#FFFFFF", "#77AADD",
"#4477AA"))
corrplot(N1, method="color", col=NULL,

```

```

type="upper", order="hclust",
addCoef.col = "black", tl.col="black", tl.srt=45,
sig.level = 0.01, insig = "blank",
diag=FALSE
)

```

В результате будет выведена корреляционная матрица Пирсона, в которой значения коэффициентов выделены цветами в соответствии с их величиной (рисунок 2).



Рисунок 13 – Пример графического представления матрицы коэффициентов корреляции Пирсона

Более подробно о возможностях визуализации корреляций с помощью пакета `corrplot` можно посмотреть по ссылке: <http://www.sthda.com/english/wiki/visualize-correlation-matrix-using-correlogram>.

Пояснения к п. 8.

Для оценки взаимосвязи между качественным признаком и количественным признаком (в случае нормального распределения значений) используется однофакторный одномерный дисперсионный анализ (ANOVA).

Функция `aov()` реализует однофакторный одномерный дисперсионный анализ:

```
aov_model <- aov(data$Experience ~ data$Group, data=data)
summary(aov_model)
```

В примере оценивается взаимосвязь между стажем и группой.

В результате выполнения функции выдается расчетное F-значение (F value), число степеней свободы (Df) и p-значение (Pr(>F))

```
              Df Sum Sq Mean Sq F value Pr(>F)
data$Group      1  765.6    765.6   1448 <2e-16 ***
Residuals    198  104.7      0.5
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Гипотеза о статистической незначимости отличий в распределении признака «стаж» по группам отвергается при уровне значимости 0,05 (p-value <<0,05). Значения стажа в группах отличаются статистически значимо.

Для оценки взаимосвязи между качественным признаком и количественным признаком (в случае, если распределение значений признака не подчиняется нормальному закону распределения) используется критерий Краскела-Уоллиса:

```
kruskal.test(data$Experience ~ data$Group, data= data)
```

В результате выдается расчетное значение критической статистики критерия (chi-squared) и p-value:

```
kruskal-wallis rank sum test

data: data$Experience by data$Group
kruskal-wallis chi-squared = 149.38, df = 1, p-value < 2.2e-16
```

Гипотеза о статистической незначимости отличий в распределении признака «стаж» по группам отвергается при уровне значимости 0,05 (p-value <<0,05). Значения стажа в группах отличаются статистически значимо.

Пояснения к п. 9.

В R реализованы различные способы построения и анализа таблиц сопряженности для исследования взаимосвязей между качественными (классификационными) признаками. В таблице 2 приведены основные функции для построения таблиц сопряженности.

Таблица 6 – Основные функции для работы с таблицами сопряженности

Функция	Описание
<code>table(var1, var2, ..., varN)</code>	Создает N-мерную таблицу сопряженности для N категориальных переменных (факторов)
<code>xtabs(formula, data)</code>	Создает N-мерную таблицу сопряженности на основе формулы и матрицы или таблицы данных
<code>prop.table(table, margins)</code>	Представляет значения таблицы в виде долей от значений крайнего поля таблицы, задаваемого параметром <i>margins</i>
<code>margin.table(table, margins)</code>	Суммирует значения таблицы по строкам или столбцам (определяется параметром <i>margins</i>)
<code>addmargins(table, margins)</code>	Вычисляет описательные статистики <i>margins</i> (суммы по умолчанию) для таблицы
<code>ftable(table)</code>	Создает компактную «плоскую» таблицу сопряженности

Для построения простой таблицы сопряженности между двумя качественными признаками используется команда:

```
table<-table(data$name_1, data$name_2)
View(table)
```

Для вычисления значений в процентах используется команда:

```
prop.table(table)
```

Для вычисления сумм по строкам и столбцам таблицы:

```
addmargins(table)
```

Для исследования взаимосвязей между качественными признаками реализованы различные тесты и коэффициенты корреляции. Один из самых часто используемых на практике – хи-квадрат критерий (тест) Пирсона. Критерий проверяет основную гипотезу об отсутствии взаимосвязи между двумя качественными признаками (например, качество документирования и пол):

```
chisq.test(table(data$Quality,data$Gender))
```

В результате выполнения функции выдается расчетное значение критической статистики критерия (X-squared), число степеней свободы (df) и р-значение:

```
Pearson's Chi-squared test

data:  table(data$Quality, data$Gender)
X-squared = 44.943, df = 3, p-value = 9.514e-10
```

Также реализован критерий Фишера для проверки основной гипотезы об отсутствии взаимосвязи между двумя качественными признаками (например, качество документирования и пол):

```
fisher.test(table(data$Quality, data$Gender))
```

В результате выполнения команды выдается р-значение и отношение шансов (отношение шансов вычисляется только для бинарных признаков и таблиц сопряженности 2 на 2). В этом случае не выводится так как признак «качество документирования» имеет четыре градации:

```
Fisher's Exact Test for Count Data

data:  table(data$Quality, data$Gender)
p-value = 6.838e-11
alternative hypothesis: two.sided
```

Ниже приведен фрагмент кода, который выполняет расчет по критерию хи-квадрат для всех качественных признаков, включенных в таблицу data, и выводит р-значение для каждой пары признаков:

```
#Получаем имена только качественных признаков
factor_names <- names(data[,unlist(lapply(data, is.factor))])

#Создаем таблицу для значений p.value
chi_results <- matrix(NA,nrow = length(factor_names),ncol=length(factor_names))
colnames(chi_results) <- c(factor_names )
rownames(chi_results) <- c(factor_names )

#Проводим Chi.test и заполняем таблицу
for (col_name in factor_names){
  for (col_name2 in factor_names){
```

```

chi_results[col_name,col_name2] <-
chisq.test(table(data[,col_name],data[,col_name2]))$p.value
}
}
chi_results

```

В результате:

	Group	Gender	Satisfaction_1	Quality
Group	1.543120e-44	5.590606e-01	9.198513e-06	1.945654e-02
Gender	5.590606e-01	1.761964e-44	1.226268e-03	9.513755e-10
Satisfaction_1	9.198513e-06	1.226268e-03	2.781632e-85	1.257279e-05
Quality	1.945654e-02	9.513755e-10	1.257279e-05	2.094185e-123

Пояснения к п. 10.

Разные варианты матричных графиков могут быть построены с помощью функции `ggpairs()` из библиотеки `GGally`. Код для построения матричных графиков между исследуемыми признаками следующий:

```

install.packages("GGally")
install.packages("ggplot2")
library(GGally)
library(ggplot2)
#Пример 1
ggpairs(data, columns = 2:5, aes(color = Group,alpha = 0.5))
#Пример 2
ggpairs(data, columns = c(2,6:10), aes(color = Group,alpha = 0.5))

```

Преимущество использования функции `ggpairs()` по сравнению с базовой функцией `pairs()` заключается в том, что построенный матричный график содержит сравнительно больше информации о распределении и взаимосвязях между признаками. На матричном графике отображена следующая информация (рисунок 3, 4):

- графики плотности распределения вероятностей для количественных признаков для каждого класса и столбиковые диаграммы для качественных признаков;
- диаграммы рассеяния между парами количественных признаков с выделением цвета класса;
- диаграммы размаха для каждого класса для количественных признаков;
- гистограммы распределения количественных признаков по классам;

– значения коэффициентов корреляции Пирсона с оценкой их статистической значимости для пар количественных признаков, рассчитанные по всем данным и с разделением по классам.



Рисунок 14 – Матричный график (Пример 1)

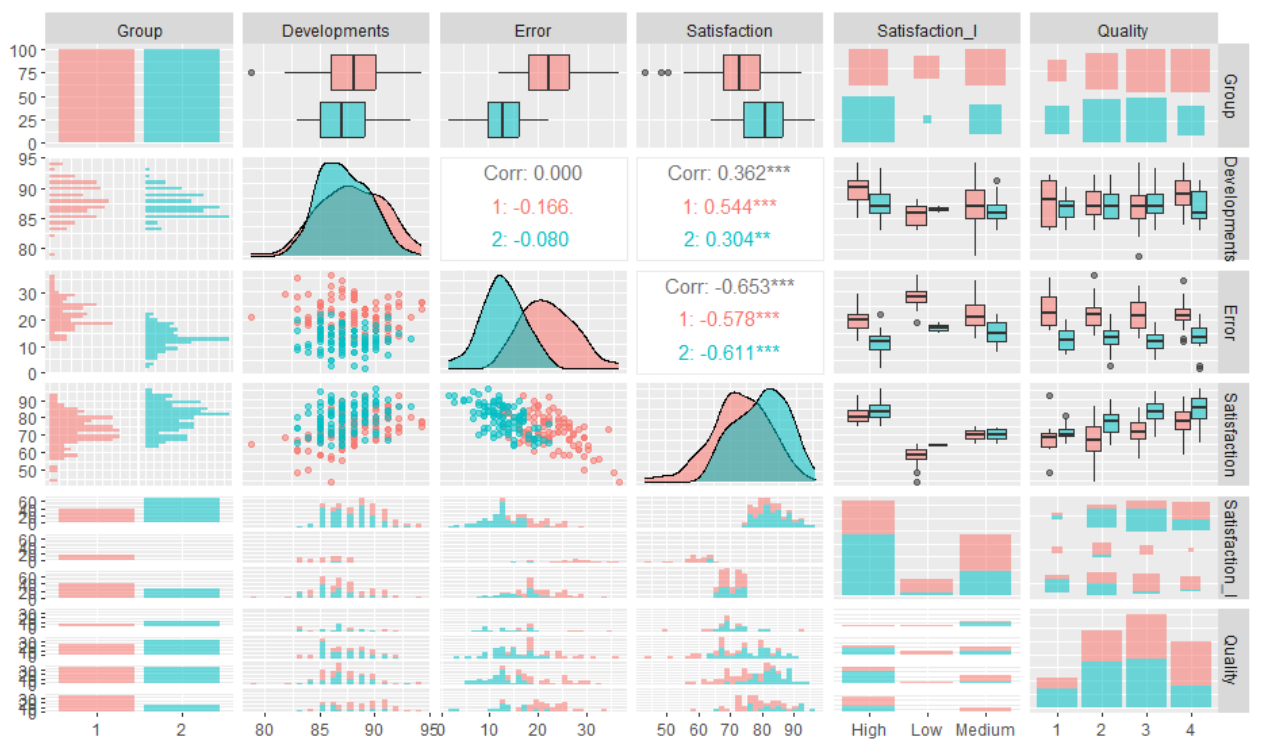


Рисунок 15 – Матричный график (Пример 2)

4. Отчет о работе

Содержание

1. Постановка задачи АД.
2. Расчет основных статистических характеристик для первой и второй группы и для всей выборки. Выводы.
3. Графический разведочный (первичный) анализ данных (код R для построения графиков и построенные графики, выводы о структуре и характере распределения данных на основе анализа графиков, различие распределения данных по группам).
4. Корреляционный анализ данных
 - проверка гипотезы о НЗР распределения значений признаков на основе критериев согласия;
 - гистограмма распределения значений признаков с наложением функции НЗР;
 - выводы о типе распределения данных;
 - матрицы коэффициентов корреляции Пирсона, Спирмена, Кендалла по группам; выводы о взаимосвязях;
 - проверка гипотезы о статистической значимости коэффициента корреляции Пирсона; выводы;
 - матрицы частных коэффициентов корреляции Пирсона по группам; выводы о взаимосвязях;
 - матричный график коэффициентов корреляции Пирсона с наложением тепловой карты (один график).
5. Исследование взаимосвязей между количественными и качественными признаками
 - результаты расчета дисперсионного анализа, критерия Краскела-Уоллиса;
 - выводы о взаимосвязях.
6. Исследование взаимосвязей между качественными признаками
 - результаты расчета критерия хи-квадрат; критерия Фишера;
 - выводы о взаимосвязях.
7. Матричные графики, построенные функцией `ggpairs()`; описание и анализ графиков.
8. Общие выводы о силе и направлении связи между исследуемыми признаками в терминах решаемой задачи для первой и второй групп).
9. Выводы по работе в произвольной форме.
10. Приложение с полным кодом на R.

5. Вопросы к работе

1. Понятие интеллектуального анализа данных. Методы Data Mining.

2. Первичный (разведочный) анализ данных. Основные задачи и методы.
3. Основные статистические характеристики и их интерпретация. Расчет основных статистических характеристик в среде R.
4. Методы графического разведочного анализа данных средствами R.
5. Какую информацию о природе данных можно получить при анализе диаграмм рассеяния, категорированных графиков, радиальных диаграмм, гистограмм, диаграмм размаха?
6. В чем достоинства и недостатки графических методов разведочного анализа данных?
7. Статистическая проверка гипотез. Логическая схема проверки гипотезы. Основные понятия: уровень значимости, надежность критерия, мощность критерия, р-значение, критическая область, область принятия гипотезы.
8. Приведите математическую постановку задачи исследования взаимосвязей между признаками. В каких предметных областях решается задача? Приведите примеры.
9. Какие методы используются для исследования закона распределения данных? В чем различия параметрических и непараметрических критериев согласия?
10. Для чего исследуются законы распределения признаков при решении задачи исследования взаимосвязей?
11. В чем различия простой и сложной гипотезы о проверке согласия теоретического и выборочного закона распределения данных?
12. Опишите основные критерии согласия для проверки гипотезы о нормальном распределении выборочных данных: алгоритм использования критерия; критическая статистика критерия; сравнительная мощность критерия.
13. Что понимается под корреляционным анализом данных? Какие основные задачи решаются в рамках корреляционного анализа данных?
14. Что понимается под ранговым корреляционным анализом данных? Какие основные задачи решаются в рамках рангового корреляционного анализа данных?
15. От каких факторов зависит выбор характеристики взаимосвязи?
16. Какие характеристики взаимосвязи используются на каждой из шкал измерения значений признака (количественная, порядковая, классификационная)?
17. Опишите основные характеристики взаимосвязи, используемые на разных шкалах: суть; основная расчетная формула; свойства; ограничения

использования; критерий для проверки гипотезы о статистической незначимости характеристики взаимосвязи; расчет в R.

18. Что понимается под дисперсионным анализом данных? Приведите постановку задачи и алгоритм применения однофакторного одномерного дисперсионного анализа. Каковы условия применения дисперсионного анализа данных?

19. Критерий Краскела-Уоллиса. Алгоритм критерия, условия применения, расчет в R, интерпретация результатов в терминах решаемой задачи.

20. Какие измерители связи применяются для измерения степени тесноты связи между качественными переменными? Их расчет в R, оценка статистической значимости и интерпретация результатов в терминах решаемой задачи.

21. Как построить разные варианты матричных графиков в R? Какие пакеты и функции используются?

Литература

1. Альсова О. К. Методы и модели решения задачи классификации. Основные этапы : учеб. пособие / О. К. Альсова, Н. А. Зеленчук. - Новосибирск: НГТУ, 2024. - 88 с. - ISBN 978-5-7782-5280-6.
2. Анализ данных: учебник для вузов / под редакцией В. С. Мхитаряна. — Москва : Издательство Юрайт, 2026. — 448 с. — (Высшее образование). — ISBN 978-5-534-19964-2. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/560311> (дата обращения: 15.01.2026).
3. Миркин, Б. Г. Базовые методы анализа данных : учебник и практикум для вузов / Б. Г. Миркин. — 3-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 297 с. — (Высшее образование). — ISBN 978-5-534-19709-9. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/560414> (дата обращения: 15.01.2026).
4. Гмурман, В. Е. Теория вероятностей и математическая статистика : учебник для вузов / В. Е. Гмурман. — 12-е изд. — Москва : Издательство Юрайт, 2026. — 479 с. — (Высшее образование). — ISBN 978-5-534-00211-9. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: <https://urait.ru/bcode/449646>
5. Кабаков Р. И. R в действии / И. Р. Кабаков ; пер. с англ. А. Н. Киселева. — Москва : ДМК Пресс, 2023. — 768 с.
6. Мاستицкий С. Э. Статистический анализ и визуализация данных с помощью R / С. Э. Мастицкий, В. К. Шитиков. — Москва : ДМК Пресс, 2023. — 497 с.

7. Официальный сайт R [Электронный ресурс]. – Режим доступа: <https://www.r-project.org> (дата обращения: 15.01.2026).
8. Репозиторий пакетов R [Электронный ресурс]. – URL: <https://cran.r-project.org/web/packages> (дата обращения: 15.01.2026).
9. Дж. Д. Лонг и Пол Титор Книга рецептов: Проверенные рецепты для статистики, анализа и визуализации данных / пер. с англ. Д. А. Беликова. – М.: ДМК Пресс, 2020. – 510 с.
10. Шитиков В. К., Мاستицкий С. Э. (2017) Классификация, регрессия, алгоритмы Data Mining с использованием R. - Электронная книга, адрес доступа: <https://github.com/ranalytics/data-mining>

Постановка задачи АД.

Варианты 1-5. Изучаются показатели работы программистов крупной организации. Для проведения статистического исследования предоставлена БД, включающая 200 записей (наблюдений), каждая из которых содержит описание показателей деятельности программиста. Все программисты разделены на две равные группы (100 записей в каждой группе). Рассматриваются следующие показатели (признаки) для каждого программиста:

- Group – группа (1, 2);
- Gender - пол (1-м, 2-ж);
- Age - возраст;
- Experience - стаж работы;
- % of on-time developments - процент разработок, выполненных в срок в рамках бюджета с требуемым функционалом (за год);
- Number of user-reported bugs - количество ошибок, выявленных пользователем (за год);
- Customer satisfaction score (score) - степень удовлетворенности заказчика (баллы);
- Customer satisfaction score (level) - степень удовлетворенности заказчика (качественная оценка);
- Documentation quality - качество документирования (1 – низкое, 2 – среднее, 3 – выше среднего, 4 – высокое).

Необходимо провести предварительный разведочный анализ данных, графический анализ данных с целью описания характера распределения и структуры данных по выделенным группам программистов, сравнения показателей по группам.

Варианты 6-10. Исследуются покупатели Интернет-магазина. Для проведения статистического исследования предоставлена БД, включающая 200 записей (наблюдений), каждая из которых содержит описание покупателя магазина. Все покупатели разделены на две равные группы (100 записей в каждой группе). Рассматриваются следующие показатели (признаки) для каждого покупателя:

- Group – группа (1, 2);
- Gender - пол (1-м, 2-ж);
- Age - возраст;
- Number of purchases - количество покупок за год;
- Average purchase price - средняя стоимость покупок за год;
- Average number of pages - среднее число страниц, просмотренных за визит;
- Number of support requests - количество обращений в службу поддержки за год;
- Participation in online surveys – участие в Интернет-опросах (1 – никогда, 2 – редко, 3 – часто, 4 – постоянно);
- Service satisfaction score (score) – степень удовлетворенности услугами (баллы);
- Service satisfaction score (level) – степень удовлетворенности услугами (качественная оценка)

Необходимо провести предварительный разведочный анализ данных, графический анализ данных с целью описания характера распределения и структуры данных по выделенным группам покупателей, сравнения покупателей по группам.

Варианты 11-15. Исследуется рынок потребителей услуги «Подключение к сети Интернет». Для проведения статистического исследования предоставлена БД, включающая 200 записей (наблюдений), каждая из которых содержит описание потребителя услуг. Все потребители разделены на две равные группы (100 записей в каждой группе). Рассматриваются следующие показатели (признаки) для каждого потребителя:

- Group – группа (1, 2);
- Gender - пол (1-м, 2-ж);
- Age - возраст;
- Work experience - стаж работы (время использования услуги);
- Average income - средний доход;
- Professional specialization - профессиональная специализация (насколько часто используется сеть Интернет в профессиональной деятельности): 1 – не использую; 2 – крайне редко; 3 – ежедневно, 4 – постоянно;
- Average number of pages - среднее количество просматриваемых страниц в месяц;
- Activity level (score) - степень активности (участие в Интернет-опросах, баллы);
- Activity level (level) - степень активности (участие в Интернет-опросах, качественная оценка);

Необходимо провести предварительный разведочный анализ данных, графический анализ данных с целью описания характера распределения и структуры данных по выделенным группам потребителей, сравнения потребителей по группам.